21/07/25

EXP-1 BRESENHAM'S LINE DRAWING ALGORITHM (|m| < 1)

AIM :-

To implement Bresenham's line drawing algo (|m| < 1) in python for drawing a line when the slope.

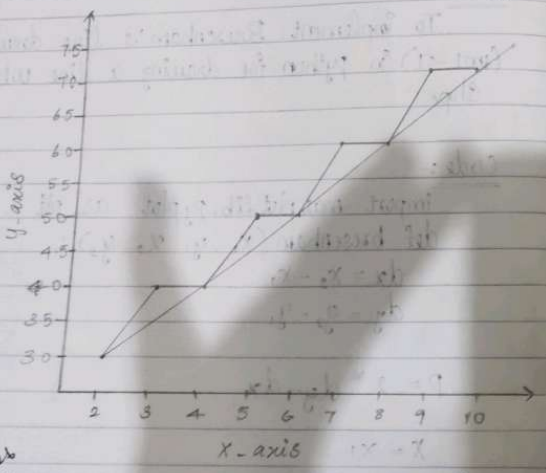
Code :-

```
import matplotlib.pyplot as plt
def bresenham(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1

    p = 2 * dy - dx

    x = x1
    y = y1
    x_points = [x]
    y_points = [y]
while x < x2:
    x += 1
    if p < 0:
        p = p + 2 * dy
    else:
        y += 1
        p = p + 2 * dy - 2 * dx
    x_points.append(x)
    y_points.append(y)
```

OUTPUT :-



o/p verified 21/7/26

(2, 2), (3, 4), (4, 4), (5.5), (6, 5), (7, 6), (8, 6), (9, 7), (10, 7).

```
plt.plot (x_points, y_points, 'bo-')
plt.title ("Bresenham Line Drawing").
plt.xlabel ("x-axis")
plt.ylabel ("y-axis").
plt.grid (True)
plt.show ()

# Example
x1, y1 = 2, 3
x2, y2 = 10, 7
bresenhmam (x1, y1, x2, y2)
```

21/7/26

Result :-

The program was executed successfully.

28/07/25

EXP-2 BRESENHAM'S LINE, CIRCLE, ELLIPSE DRAWING ALGORITHM. (|m|<1)

AIM :-

To implement Bresenham's line, circle, Ellipse drawing algorithm in python.

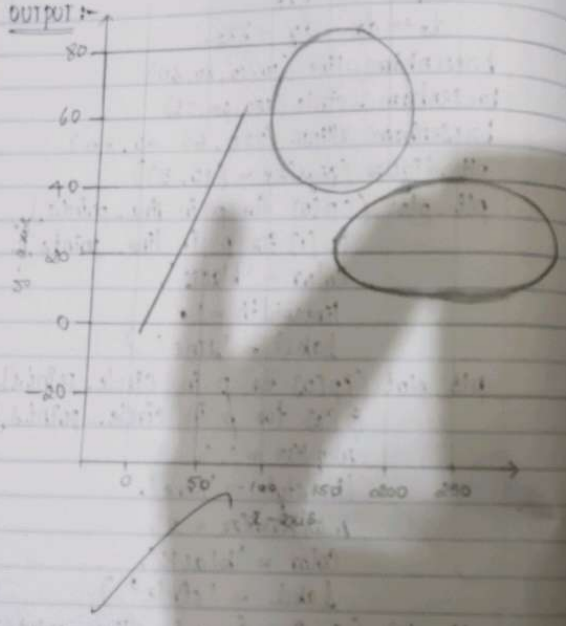
Code :-

```
import matplotlib.pyplot as plt
line_points = []
def plot_line(x,y):
    line_points.append((x,y))
def bresenham_line(x1,y1,x2,y2):
    dx = abs(x2-x1)
    dy = abs(y2-y1)

    sx = 1 if x2>x1 else -1
    sy = 1 if y2>y1 else -1
    err = dx-dy.
while True:
    plot_line.(x1,y1)
    if x1==x2 and y1==y2:
        break.
    e2 = 2*err
    if e2 > -dy:
        err -=dy
        x1 +=sx.
    if e2 < dx:
        err +=dx
        y1 +=sy.
```

```
plt.title("Bresenham Line, Circle and Ellipse Drawing")
plt.xlabel("x-axis")
plt.ylabel("Y-axis")
plt.axis("equal")
plt.grid(True)
plt.legend()
plt.show().
```

OUTPUT :-



DDA Algorithm

4/8/26

RESULT :-

The program was executed successfully

04/08/26

EXP-3 : DIGITAL DIFFERENTIAL ANALYZER (DDA) LINE DRAWING ALGORITHM

Aim :-

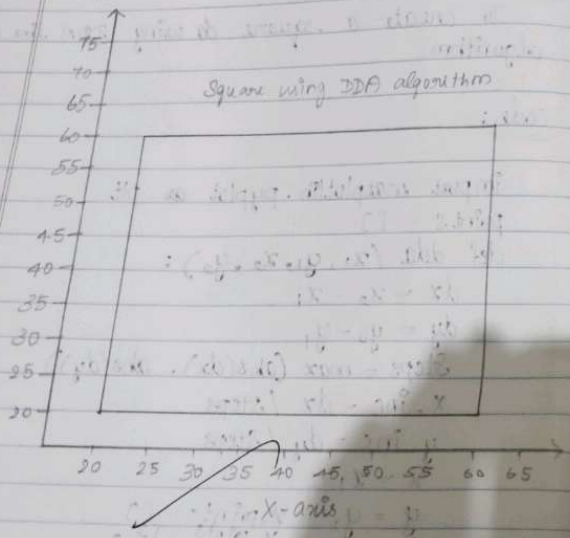
To create a square ds using DDA line drawing algorithm.

code :-

```
import matplotlib.pyplot as plt
points = []
def dda (x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    Steps = max (abs(dx), abs(dy))
    x_inc = dx / steps
    y_inc = dy / steps.
    x = x1
    y = y1      x_points = []
                y_points = []
    for i in range (steps + 1):
        points. append ((round(x), round(y)))
        x += x_inc.
        y += y_inc.
x = 20
y = 20
side = 40

plt. plot (x_points, y_points, 'k', linewidth=2)
```

OUTPUT:-




```
dda (x, y, x+side, y)
dda (x+side, y, x+side, y+side)
dda (x+side, y+side, x, y+side)
dda (x, y+side, x, y)

plt.figure(figsize=(5,5))
plt.plot([p[0] for p in points],
         [p[1] for p in points],
         'k', markersize=3)

plt.title("Square using DDA Algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.gca().set_aspect('equal')
plt.show()
```

4/8/24

RESULT:-

Hence the program is verified successfully.

04/08/26

EXP-4 BUILD AN OBJECT USING BRESENHAM ALGORITHM.

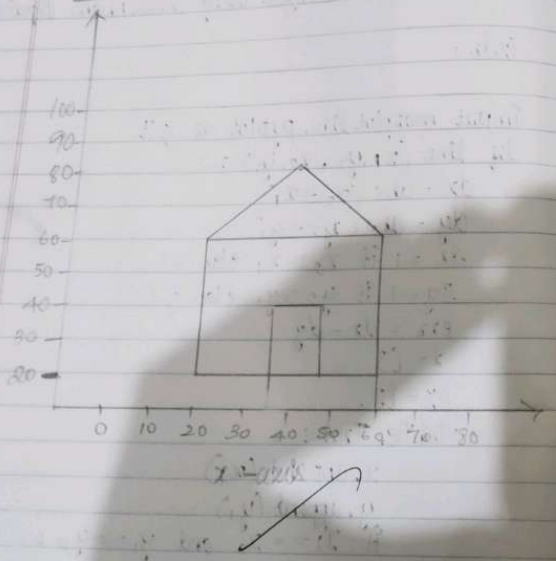
Aim :-

To build an object using Bresenham algorithm.

Code :-

```
import matplotlib.pyplot as plt
def line (x1, y1, x2, y2):
    dx = abs (x2 - x1)
    dy = abs (y2 - y1)
    sx = 1 if x2 > x1 else -1
    sy = 1 if y2 > y1 else -1
    err = dx - dy
    x = []
    y = []
    while True:
        x.append (x1)
        y.append (y1)
        if x1 == x2 and y1 == y2:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy
            x1 += sx
        if e2 < dx:
            err += dx
            y1 += sy
```

OUTPUT




```
plt.plot (x,y,'k')

line (20,20,20,60)
line (20,60,60,60)
line (60,60,60,20)
line (60,20,20,20).
line (20,60,40,80)
line (40,80,60,60).
line (35,20,25,40)
line (45,20,45,40)
line (35,40,45,40)
plt.title("House using Bresenham Algorithm")
plt.axis("equal").
plt.grid(True)
plt.show().
```

4/8/20

RESULT :-

Hence the program is verified Successfully

11/8/26

EXP-5 TRAFFIC SIGNAL USING BRESENHAM'S ALGORITHM

Aim:-

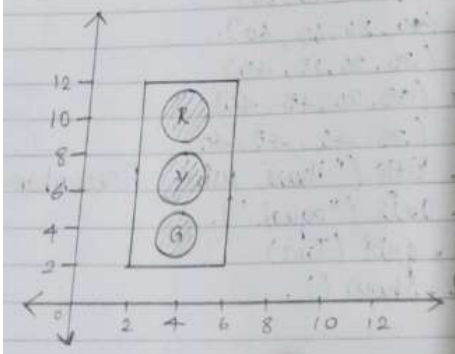
To implement the program to draw the traffic signal using bresenham's algorithm.

Code:-

```
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Rectangle
fig, ax = plt.subplots()
box = Rectangle((2,1), 4, 10, facecolor="black")
ax.add_patch(box)
red = Circle((4,8), 1, facecolor="red",
             edgecolor="white")
ax.add_patch(red)
yellow = Circle((4,6), 1, facecolor="yellow",
                edgecolor="white")
ax.add_patch(yellow)
green = Circle((4,4), 1, facecolor="green",
               edgecolor="white")
ax.add_patch(green)
ax.set_xlim(0,8)
ax.set_ylim(0,12)
ax.set_aspect("equal")
ax.grid(True)
plt.title("Traffic Signal")
plt.show()
```

RESULT:- Hence the code is verified successfully

OUTPUT:-



11/8/26

EXP-6 RECTANGLE USING BRESENHAM'S ALGORITHM

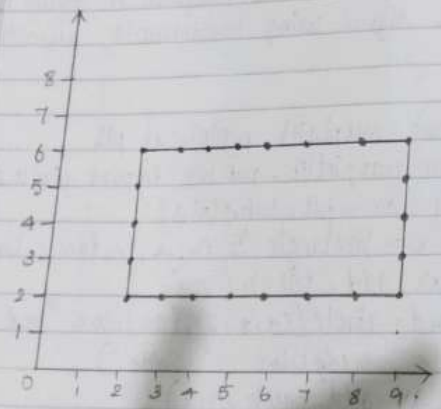
Given :- Width = 8, Height = 5, vertex (2,2)

Aim :- To implement the rectangle using the bresenham's algorithm in python

code :-

```
Import matplotlib.pyplot as plt
def bresenham (x0, y0, x1, y1):
p = [];
dx = abs (x1 - x0)
dy = abs (y1 - y0)
sx = 1
if x0 < x1,
else -1; sy = 1 if y0 < y1, else -1
e = dx - dy
while 1:
    p. append (x0, y0)
if (x0, y0) == (x1, y1): retarn p.
e2 = 2*e
if e2 > -dy : e -= dy ; x0 += sx.
if e2 < dx : e += dx ; y0 += sy
x, y, w, h = 2, 2, 8, 5.
xe, ye = x + w - 1, y + h - 1.
for a, b, c, d in [(x, y, xe, y), (x, ye, xe, ye),
        (xe, y, xe, ye)]:
p = bresenham (a, b, c, d)
```

OUTPUT :-



11/8/24

EXP-7: BUTTERFLY USING ELLIPSE

Aim :-
To implement the program to draw a butterfly using ellipse

code :-

```
import matplotlib.pyplot as p
import math
def e(cx, cy, rx, ry, c='b')
for i in range(361):
    t = math.radians(i)
    p.plot(cx + rx * math.cos(t), cy + ry *
                math.sin(t), c)
for x in [2.5, 7.5]:
    e(x, 5.2, 2.5)
    e(x, 2.5, 1.5, 1.5)
    e(5, 4.5, 1.8, 2.5, 'k')
    e(5, 7, .5, .5, 'k')
p.plot([4.8, 4, 3.5], [7.4, 8, 8.3], 'k')
p.plot([5.2, 6, 6.5], [7.4, 8, 8.3], 'k')
p.axis('equal')
p.grid()
p.show()
```

RESULT :-
The program to draw a butterfly has been successfully done.

26/3/24

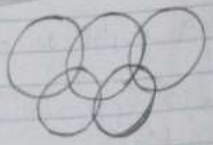
EXP-8 : OLYMPIC LOGO USING ELLIPSE

Aim :-
To implement the program to draw a olympic logo using ellipse.

code :-

```
import matplotlib.pyplot as plt
import math
def ellipse (cx, cy, rx, ry):
t = [math.radians(i) for i in range(361)]
return [cx + rx * math.cos(a) for a in t],
       [cy + ry * math.sin(a) for a in t]
rings = [(2.4, 'b'), (5.4, 'k'), (8.4, 'r'),
         (3.5, 2.5, 'y'), (6.5, 2.5, 'g')]
for x,y, c in rings :
  x, y = ellipse (x, y, 1.5, 2)
  plt.plot (x, y, c, lw = 3)
plt.axhline (0, color = 'k')
plt.axvline (0, color = 'k')
plt.grid ()
plt.xlim (0, 10)
plt.ylim (0, 6)
plt.xticks (range (11))
plt.yticks (range (4))
plt.axis ('equal')
plt.title ("olympic logo - DDA")
plt.show ()
```

OUTPUT :-



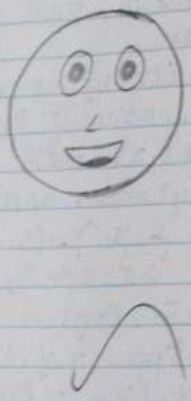
RESULT :- Olympic logo using ellipse is done successfully

25/8/24

EXP-9 : SMILING FACE WITH EYES, NOSE, MOUTH USING BRESENHAM'S ALGORITHM.

Aim :- To implement the program to draw a smiling face with eyes, nose, mouth using Bresenham's algorithm.

OUT :-



Code :-

```
Import matplotlib.pyplot as plt
Import math
def c(x,y,r,col):
p=[(x+r*math.cos(math.radians(a)),
   (y+r*math.sin(math.radians(a)))
for a in range(361)]
x,y = zip(*p); plt.fill(x,y,col); plt.plot(x,y,'k')
c(5.3,3,'yellow')
for x in [3.8, 6.2]:
  c(x,6.2,.5,'white')
  c(x,6.2,.25,'black')
plt.plot([5,4.7,5.2],[5.8,5.1,5.1],'k',lw=2)
x=[3.3+i*.05 for i in range(41)]
i=[4.1+.25*math.sin(math.pi*i/40)
    for i in range(41)]
plt.fill(x,y,'red');
plt.plot(x,y,'k',lw=2)
x=[4+i*.1 for i in range(21)]
plt.plot(x,y,'pink',lw=4)
plt.axis('equal')
```

08/09/25

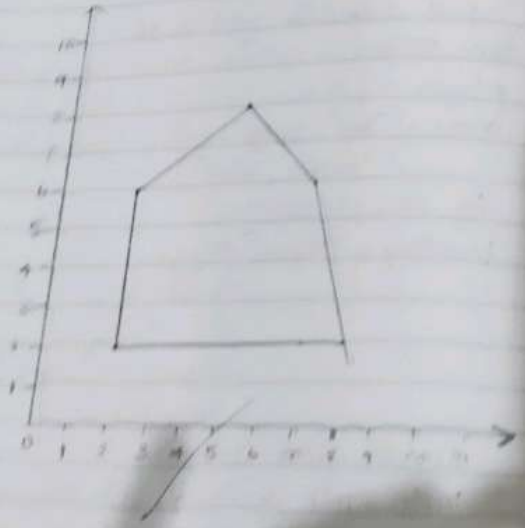
EXP-10: IMPLEMENT SCAN LINE CONVERSION OF POLYGON & USE FLOOD FILL ALGORITHM

Aim :- To implement a scan line conversion of polygon & use flood fill algorithm

code :-

```
import matplotlib.pyplot as plt
from matplotlib.patches import Polygon, Rectangle
polygon = [(2,2), (8,2), (7,6), (5,8), (2,6)]
colors = ['red', 'orange', 'yellow', 'green', 'blue',
          'purple', 'pink']
fig, ax = plt.subplots()
boundary = Polygon( polygon,
                    closed=True,
                    fill = False,
                    edgecolor = 'black',
                    linewidth = 2 )
ax.add_patch (boundary)
ymin = min (y for x, y in polygon)
ymax = max (y for x, y in polygon)
for y in range (ymin, ymax):
    intersections = []
for i in range (len(polygon)):
    x1, y1 = polygon[i]
    x2, y2 = polygon[(i+1) % len (polygon)]
    if y1 != y2 and min (y1, y2) <= y < max (y1, y2):
    x = x1 + (y - y1) * (x2 - x1) / (y2 - y1)
    intersections.append (x).
Intersections.sort ()
```

OUTPUT



```
color = colors[(y - ymin) % len(colors)]
for i in range(0, len(intersections), 2):
    if i+1 < len(intersections):
        x_left = intersections[i]
        x_right = intersections[i+1]
        rect = Rectangle((x_left, y),
                         x_right - x_left,
                         1,
                         facecolor = color,
                         edgecolor = None)
        rect.set_clip_path(boundary)
        ax.add_patch(rect)
x = [p[0] for p in polygon] + [polygon[0][0]]
y = [p[1] for p in polygon] + [polygon[0][1]]
ax.plot(x, y, 'black', linewidth = 2)
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)
ax.set_xticks(range(11))
ax.set_yticks(range(11))
ax.set_aspect('equal')
ax.grid(True)
plt.title("Scan Line Conversion - Polygon Filling")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

RESULT:

The program to implement the scan line conversion polygon filling is done successfully.

15/09/20

EXP-11 : DRAW A BASIC SHAPE & APPLY TRANSFORMATIONS LIKE ROTATION, TRANSLATION & SCALING

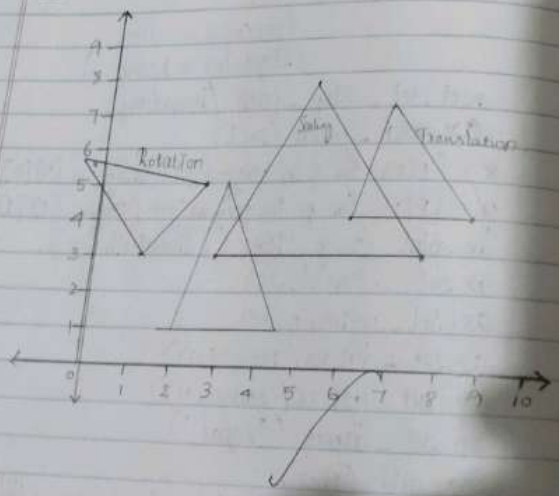
AIM :-

To implement a transformations to a basic shape

code :-

```
import matplotlib.pyplot as plt
import numpy as np
p= np.array([[2,2],[5,2],[3,5],[2,2]])
t= p+ [4,2]
s = p * 1.5
a = np.radians(45)
R= np.array([[np.cos(a), -np.sin(a)],
             [np.sin(a), np.cos(a)]])
r= p @ R.T
plt.plot(p[:,0], p[:,1], label = "original")
plt.plot(t[:,0], t[:,1], label = "Translation").
plt.plot(s[:,0], s[:,1], label = "Scaling")
plt.plot(r[:,0], r[:,1], label = "Rotation")
plt.axhline(0)
plt.axvline(0)
plt.grid()
plt.legend()
plt.show()
```

OUTPUT :-



RESULT :-

Hence it is verified successfully

23/09/24

EXP-12 DRAW A RECTANGLE & APPLY TRANSFORMATION

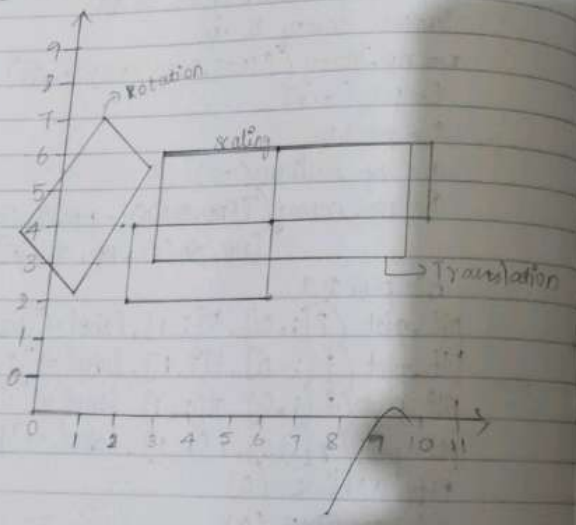
Aim:-

To draw a rectangle & apply transformation.

Code :-

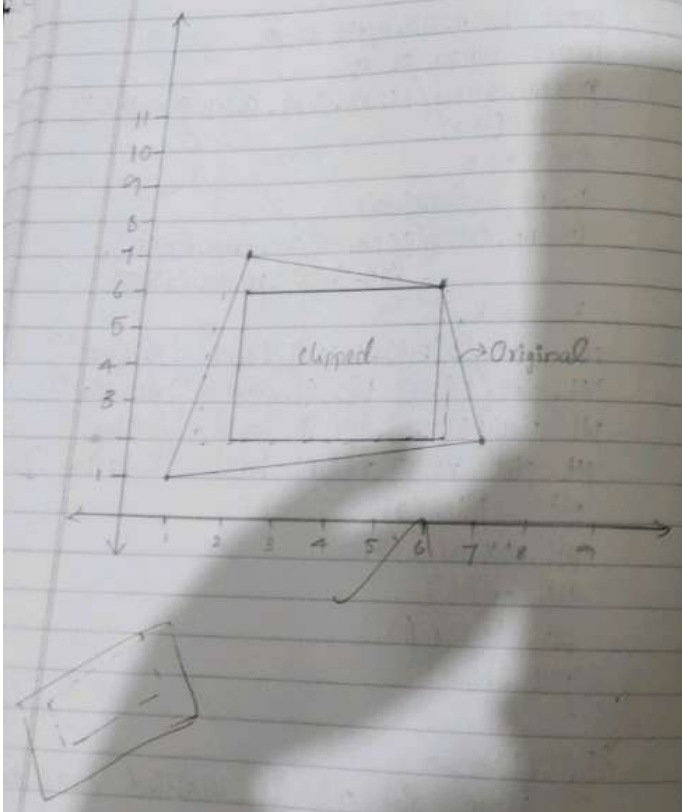
```
import matplotlib.pyplot as plt
import numpy as np
P = np.array([[2,2],[6,2],[6,4],[2,4],[2,2]])
t = p + [4,2]
S = p * 1.5
a = np.radians(45)
R = np.array([[np.cos(a), -np.sin(a)],
              [np.sin(a), np.cos(a)]])
r = p @ R.T
plt.plot(p[:,0], p[:,1], label = "original")
plt.plot(t[:,0], t[:,1], label = "Translation")
plt.plot(S[:,0], S[:,1], label = "scaling")
plt.plot(r[:,0], r[:,1], label = "Rotation")
plt.axhline(0)
plt.axvline(0)
plt.grid()
plt.legend()
plt.show()
```

OUTPUT:-



RESULT:-

Hence, the code is verified successfully.



22/01/26

EXP-13 POLYGON CLIPPING USING SUTHERLAND HODGMAN

Aim:-

To clipping the polygon using sutherland hodgman

code:-

```
import matplotlib.pyplot as plt
p=[(1,1),(1,2),(5,5),(2,1)]
def clip(p, xmin, xmax, ymin, ymax):
    return [(max(xmin, min(xmax, x)),
             max(ymin, min(ymax, y))) for x, y in p]
c=clip(p,2,6,2,6)
x,y=zip(*(p+[p[0]]))
plt.plot(x,y,label="Original")
x,y=zip(*(c+[c[0]]))
plt.plot(x,y,label="Clipped")
plt.grid()
plt.legend()
plt.show()
```

RESULT:-

Hence, the code is verified successfu

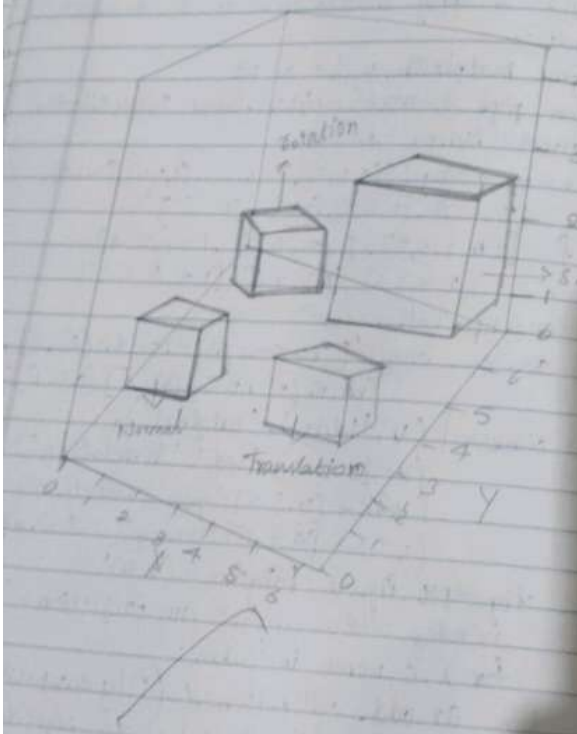
29/09/26

EXP-14: DRAW A 3D SHAPE & APPLY TRANSFORMATION

Aim:- To apply transformation on a 3D object

code:-

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.import Poly3DCollection
import numpy as np
V = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],
    [0,0,1],[1,0,1],[1,1,1],[0,1,1]])
f = [[0,1,2,3],[4,5,6,7],[0,1,5,4],
     [2,3,7,6],[0,3,7,4],[1,2,6,5]]
a = np.radians(30)
R = np.array([[np.cos(a),-np.sin(a),0],
              [np.sin(a),np cos(a),0],[0,0,1]])
cubes=[(V,'blue','original'),
       (V+[3.0,0],'red','Translation'),
       (V@R+[0,3,0],'green','Rotation'),
       (V*2+[3,3,0],'Orange','Scaling')]
fig=plt.figure.(figsize=(8,6))
ax=fig.add_subplot(111, projection='3d')
for x,c,name in cubes:
    ax.add_collection3d(Poly3DCollection([[x[i]
    for i in f], facecolors=c, edgecolors='black',
                alpha=.7))
    p=x.mean(axis=0)
    ax.text(*p,name)
```




```
ax.set_xlim(-1,6)
ax.set_ylim(-1,6)
ax.set_zlim(-1,4)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
plt.show()
```

RESULT :-

Hence the code is successfully verified.

10/6/26

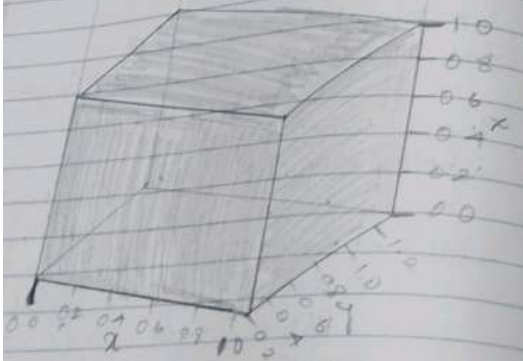
EXP-15 : CREATE & RENDER 3D OBJECTS USING POLYGONS & APPLY BASIC COLOR AND SHADING TECHNIQUES.

Aim :-

To creat & render 3D objects & apply shading & coloring.

code :-

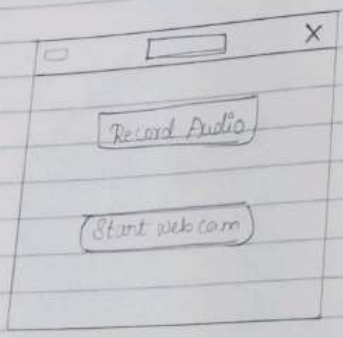
```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import
                    poly3DCollection
import numpy as np
vertices = np.array([[0,0,0], [1,0,0], [1,1,0],
        [0,1,0], [0,0,1], [1,0,1], [1,1,1], [0,1,1]])
faces = [vertices[i] for i in [0,1,2,3]],
        vertices[i] for i in [4,5,6,7]],
        vertices[i] for i in [0,1,5,4]],
        vertices[i] for i in [2,3,7,6]],
        vertices[i] for i in [1,2,6,5]],
        vertices[i] for i in [4,7,3,0]] ]
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
cube = poly3DCollection(faces, alpha=0.7,
    facecolors=['red','green','blue','yellow','orange',
                'purple'], edgecolor='black',)
ax.add_collection3d(cube)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('X')
```



```
ax.set_xlim(0,1)
ax.set_ylim(0,1)
ax.set_zlim(0,1)
plt.show()
```

RESULT :-

Hence the code is verified successfully



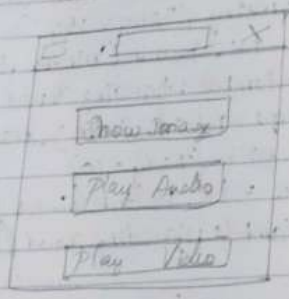
12/10/25

EXP – 16 : CAPTURE VIDEO / AUDIO FROM WEBCAM OR MICROPHONE AND DISPLAY ON MULTIMEDIA INTERFACE

Aim :- To develop a program that capture video, audio from the webcam and microphone

code :-

```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
import tkinter as tk
def record_audio():
    duration = 5
    fs = 44100
    print("Recording Audio...")
    audio = sd.rec(int(duration * fs),
        samplerate = fs, channels = 2)
    sd.wait()
    write("recorded_audio.wav", fs, audio)
    print("Audio saved Successfully!")
def capture_video():
    cap = cv2.VideoCapture(0)
    print("Press 'q' to stop video.")
while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("Webcam Feed", frame)
```



10/10/26

EXP-17 : MULTIMEDIA APP INTEGRATES IMAGES, SOUND & VIDEO IN UI

Aim :- To develop a simple multimedia app that integrates images, video & audio in UI

Code :

```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading
def image():
img = Image.open("sample_image.jpg").resize((200,200))
photo = ImageTk.PhotoImage(img)
panel.config(image = photo)
panel.image = photo
def audio():
playsound("sample_audio.mp3")
def video():
cap = cv2.VideoCapture("sample_video.mp4")
while cap.isOpened():
ret, frame = cap.read()
if not ret:
break
cv2.imshow("Video", frame)
if (cv2.waitKey(25) & 0xFF == ord('q'):
break
cap.release()
```